Page 1

JENKINS

COMPLETE NOTES

(Jenkins)



BEGINNER TO ADVANCED

Manjunatha HG

Jenkins CI/CD Complete Notes

1 What is Jenkins?

- Jenkins is an open-source automation server used to automate CI/CD pipelines.
- It helps automate building, testing and deploying applications.

2 Why Jenkins is used?

- Automates software development lifecycle from build to deployment.
- Reduces manual work and human errors.
- Speeds up delivery with consistent & repeatable processes.
- Highly extensible with rich plugin ecosystem.

3 CI vs CD vs Continuous Deployment

CI	CD (Delivery)	CD (Deployment)
• Developers integrate code frequently. • Automatically build and test. • Goal: Detect issues early.	• Code is always ready to release. • Requires manual approval to deploy. • Goal: Ensure release is reliable.	• Automatic deployment to production after testing. • No manual approval needed. • Goal: Ship faster and safer.

4 Benefits of Jenkins

- ☑ Open Source & Free
- ☑ Easy to Install & Use
- ☑ Huge Plugin Ecosystem
- ☑ Scalable (Distributed Builds)
- ☑ Supports Multiple Tools
- ☑ Cross-platform
- ☑ Strong Community Support
- ☑ Improves Developer Productivity
- ☑ Enables CI/CD Best Practices

☆ Interview Tip

Jenkins is the most popular automation server for CI/CD and is widely used in DevOps pipelines to deliver software faster and reliably.

5 Real-world Workflow Overview



Developer	Git Repository	Jenkins	Build	Test	Deploy
Write Code & Commit	Code Stored in Git (GitHub/Gitlab)	Webhook Triggers Jenkins Job	Compile Code & Build Artifacts	Run Automated Tests (JUnit/TestNG)	Deploy to Staging/ Production

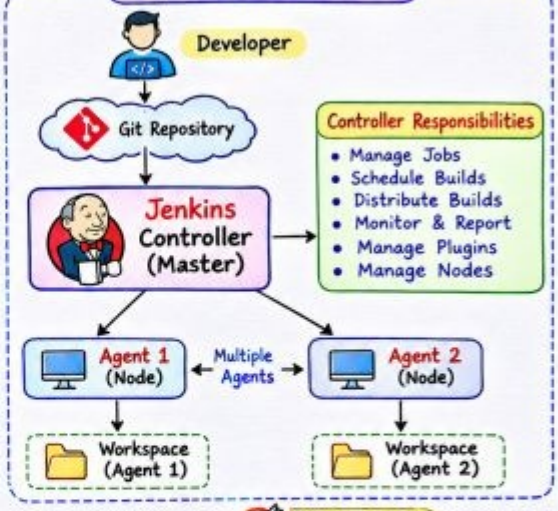
Feedback Loop for Continuous Improvement

Jenkins Architecture

Jenkins Components

1. **Controller (Master):**
   - Manages jobs, schedules builds, distributes work to agents and stores configurations.
2. **Agent (Node):**
   - Executes builds on behalf of the controller.
   - Can be one or multiple.
3. **Executors:**
   - Slots on controller or agents that run builds.
   - Each executor runs one build at a time.
4. **Jobs:**
   - Tasks defined in Jenkins (Freestyle or Pipeline) that perform build, test, deploy, etc.
5. **Workspace:**
   - Directory on the agent where code is checked out and build activities happen.

Architecture Overview



How it Works?

1. Developer pushes code to Git Repository.
2. Webhook or Polling triggers Jenkins.
3. Controller schedules the job and assigns it to an available agent.
4. Agent checks out the code in its workspace.
5. Build, Test, Package, Deploy steps are executed.
6. Results are sent back to Controller and shown in Dashboard.

Key Points

- ✓ Controller does not run builds (unless no agent available).
- ✓ Agents can be on same machine or remote.
- ✓ Supports distributed builds for scalability.
- ✓ High availability can be achieved with multiple controllers.

★ **Note:** For better performance, run builds on Agents, not on Controller.

Manjunatha HG

Installation & Initial Setup

Jenkins is a self-contained automation server. Below are the steps to install and set it up using Docker.



Jenkins

1 Java Requirement

- Jenkins requires Java 17 or newer (LTS recommended).

```
$ java -version
openjdk version "17.0.x" 2024-xx-xx
OpenJDK Runtime Environment
OpenJDK 64-Bit Server VM
```

Java

2 Install Jenkins (Using Docker)

- Docker is the easiest and recommended way to install Jenkins.

```
$ docker run -d \
  --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  jenkins/jenkins:lts
```

docker

3 Access Jenkins

- Open your browser and go to: `http://localhost:8080`
- You should see the Jenkins Unlock page.

4 Unlock Jenkins

- Get the initial admin password from container logs or from the file.

```
$ docker logs jenkins
        or
$ docker exec jenkins cat
  /var/jenkins_home/secrets/
  initialAdminPassword
```

http://localhost:8080

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to:

/var/jenkins_home/secrets/initialAdminPassword

Please copy the password and paste it below.

Administrator password

Continue

5 Install Suggested Plugins

- Select "Install suggested plugins" to install the core plugins.

Customize Jenkins

Plugins extend Jenkins with additional features.

Install suggested plugins | Select plugins to install

Plugins

6 Create Admin User

- Add the first admin user who will have full access.

Create First Admin User

Username	admin
Password	*******
Full Name	Rakesh Kumar
Email	rakesh@example.com

Save and Continue

Admin User

7 Jenkins Dashboard

- Once setup is complete, you will land on the Jenkins Dashboard.

Jenkins Search ... admin

New Item
People
Build History
Manage Jenkins
My Views

Welcome to Jenkins!

Jenkins is ready to automate your CI/CD!

Dashboard

Tips

- Use Jenkins LTS image for stability.
- Map ports 8080 (UI) and 50000 (Agent communication).
- Store data in a volume for persistence in production.

Important Note

In production, configure reverse proxy (Nginx/Apache), SSL, security realm, and backups.

Common Mistakes

- Not opening required ports.
- Not saving initial admin password.
- Using outdated Jenkins or plugins.

Manjunatha HG

Jenkins Dashboard & Jobs

Page 5

Jenkins provides a web UI to manage jobs, builds and CI/CD pipelines efficiently.

1 Dashboard
- Home page that shows all jobs, build status and system info.

2 Freestyle Job
- Traditional job type. Configured step-by-step through UI.

3 Pipeline Job
- Defined using a Jenkinsfile. Supports complex CI/CD workflows.

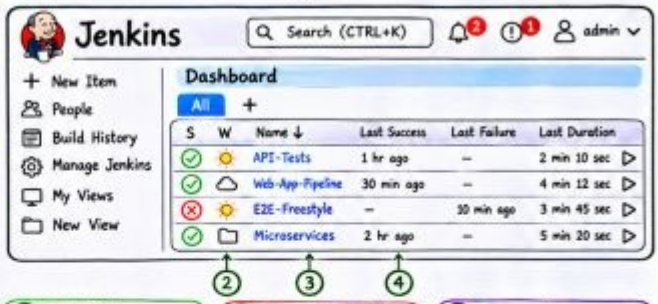
4 Folder
- Organize jobs into folders for better structure & access control.

5 Build History
- Shows all past builds with status, duration & result.

6 Console Output
- Detailed log of what happened during the build.

7 Parameters
- Allow dynamic inputs while triggering the job.

8 Workspace
- Directory on the node/ agent where code is checked out and build runs.

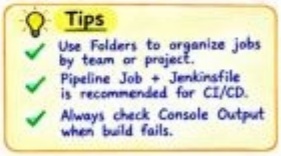












Job Types Quick View

Type	Configuration	Use Case	Version Control
Freestyle Job	UI (Step by Step)	Simple builds, quick tasks	Optional
Pipeline Job	Jenkinsfile (Code)	CI/CD, complex workflows	Recommended
Multibranch Pipeline	Jenkinsfile in Repo	Branch based CI/CD	Required

Quick Start (Docker)

```
# Java check
# java -version
# Run Jenkins with Docker
# docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts
```

Manjunatha HG

Jenkins Pipeline Fundamentals

1 Pipeline

- A Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins.

2 Pipeline as Code

- Pipelines are defined in a Jenkinsfile which is stored in source control.
- This enables versioning, reusability and collaboration.

3 Jenkinsfile

- A text file (Jenkinsfile) written in Groovy DSL that defines the entire CI/CD workflow.

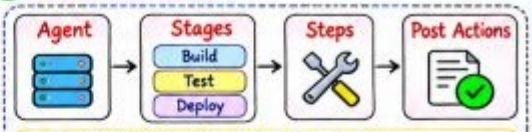
4 Declarative Pipeline

- A structured and simplified syntax with predefined sections. Recommended for most use cases.

5 Scripted Pipeline

- A more flexible approach using Groovy code. Useful for complex logics and dynamic behavior.

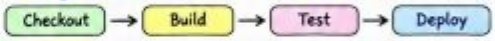
6 Pipeline Anatomy



- Pipeline executes on an Agent through Stages.
- Each Stage contains one or more Steps.

7 Pipeline Stages

- Stages represent major phases in the CI/CD process (e.g., Build, Test, Deploy).
- Stages can run sequentially or in parallel.



8 Steps

- Steps are the individual tasks performed inside a Stage. Examples: echo, sh, checkout, junit, archiveArtifacts, etc.

9 Agent Block

- Specifies where the Pipeline or Stage will run. Can be any available node/agent.



Declarative Pipeline Example

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Hello from Jenkins Pipeline!'
      }
    }
  }
}
```

- Run this pipeline on any available agent.
- Define all the stages in the pipeline.
- One stage: Build
- Steps are tasks executed inside the stage.
- A simple step to print message in console.

Key Takeaways

- ✓ Pipelines are written as Code (Jenkinsfile).
- ✓ Declarative syntax is simple and recommended.
- ✓ Scripted syntax is more flexible.
- ✓ Pipelines are made of Stages and Steps.
- ✓ Agent defines where the pipeline runs.
- ✓ Easier version control, review and reuse of CI/CD workflows.

Interview Tip

- Always prefer Declarative Pipeline unless you need advanced logic that requires Groovy scripting.

Remember

- Jenkinsfile = Pipeline as Code.
- Stages = Phases (Build, Test, Deploy).
- Steps = Tasks inside a stage.

Common Use

- Build, Test, Code Quality,
- Package, Deploy, Notify.

Manjunatha HG



Complete Jenkinsfile (Production Pipeline)

Production CI/CD Pipeline Flow

1. **Checkout** — Checkout source code from SCM (Git).
2. **Build** — Compile the code using build tool (Maven/Gradle).
3. **Unit Test** — Run unit tests (JUnit/TestNG).
4. **Code Quality** — Analyze code quality using SonarQube.
5. **Package** — Package the application (JAR/WAR).
6. **Docker Build** — Build Docker image from Dockerfile.
7. **Push Image** — Push Docker image to registry (Docker Hub/ECR).
8. **Deploy** — Deploy to environment (Kubernetes/Server).
9. **Notify** — Send notifications (Slack/Email).

Sample Jenkinsfile (Declarative Pipeline)

```
pipeline {
    agent any
    environment {
        REGISTRY = "docker.io/rakesh123"
        IMAGE_NAME = "myapp"
        IMAGE_TAG = "${env.BUILD_NUMBER}"
    }
    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package -DskipTests'
            }
        }
        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }
        stage('Code Quality') {
            steps {
                withSonarQubeEnv('sonar') {
                    sh 'mvn sonar:sonar'
                }
            }
        }
        stage('Package') {
            steps {
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
        stage('Docker Build') {
            steps {
                sh "docker build -t ${REGISTRY}/${IMAGE_NAME}:${IMAGE_TAG} ."
            }
        }
        stage('Push Image') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'docker-cred',
                                                  usernameVariable: 'DOCKER_USER',
                                                  passwordVariable: 'DOCKER_PASS')]) {
                    sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin'
                    sh "docker push ${REGISTRY}/${IMAGE_NAME}:${IMAGE_TAG}"
                }
            }
        }
        stage('Deploy') {
            steps {
                sh 'kubectl apply -f k8s/deployment.yaml'
            }
        }
        stage('Notify') {
            steps {
                slackSend channel: '#ci-cd', color: 'good',
                          message: "✅ Build ${env.BUILD_NUMBER} Success!"
            }
        }
    }
    post {
        always {
            cleanWs()
        }
        failure {
            slackSend channel: '#ci-cd', color: 'danger',
                      message: "❌ Build ${env.BUILD_NUMBER} failed!"
        }
    }
}
```

Pipeline Benefits

- ✓ Automated end-to-end delivery.
- ✓ Consistent & repeatable builds.
- ✓ Early feedback & quality assurance.
- ✓ Faster releases with less manual effort.
- ✓ Easy rollback & traceability.

Best Practices

- ✓ Keep Jenkinsfile in SCM.
- ✓ Use meaningful stage names.
- ✓ Fail fast, fail early.
- ✓ Use credentials securely.
- ✓ Keep pipeline modular and reusable.

Remember

- Pipeline as Code.
- Version Controlled + Reusable.
- Collaborative + Scalable.

Common Mistakes

- Hardcoding credentials.
- Skipping unit tests.
- Not handling failures.
- Huge Jenkinsfile (keep it modular).

Manjunatha HG

Integrating Jenkins with DevOps Tools



Jenkins acts as the orchestrator that connects and automates tools across the CI/CD lifecycle.

CI CD

Build → Test → Deploy

1 Git (SCM)
- Jenkins pulls source code from Git repositories using plugins like Git plugin.
- Supports branches, tags and webhooks.

2 GitHub / GitLab / Bitbucket
- Jenkins integrates via webhooks for build triggers, PR checks, commit status updates and repository management.

3 Maven
- Jenkins uses Maven to build Java projects.
- Handles dependencies, compilation and packaging.

4 Gradle
- Build automation for Java, Kotlin and Groovy projects.
- Supports plugins, tasks and dependencies.

5 NPM
- Jenkins runs NPM commands for frontend projects.
- Install dependencies, build, test and create artifacts.

6 Docker
- Build Docker images, run containers, tag and push to Docker Hub/Registry using Docker Pipeline.

7 Kubernetes
- Jenkins deploys applications to Kubernetes using kubectl or Kubernetes plugin.
- Manages deployments, pods, services and rollbacks.

8 SonarQube
- Jenkins sends code to SonarQube for static code analysis.
- Quality Gate result can pass/fail the build.

9 Nexus
- Jenkins publishes artifacts (Maven/Gradle/NPM) to Nexus repository.
- Nexus is used for storage and versioning.

10 Artifactory
- Store and manage build artifacts, Docker images and dependencies in Artifactory.

11 Slack
- Jenkins sends notifications to Slack channels for build status, success, failure and deployment updates.

12 Email
- Jenkins can send email notifications using Email Extension plugin for build results and alerts.

13 Webhook
- Tools trigger Jenkins jobs using Webhooks (push code, PR, issue, release etc.).
- Enables real-time CI/CD automation.

Key Benefits
- ✓ Automated end-to-end CI/CD workflow.
- ✓ Toolchain integration and orchestration.
- ✓ Faster delivery with fewer manual steps.
- ✓ Improved quality and visibility.
- ✓ Scalable and extensible DevOps pipeline.

Best Practices
- ✓ Use webhooks for instant triggers.
- ✓ Store secrets in Jenkins Credentials.
- ✓ Fail fast on quality gate failures.
- ✓ Keep tools & plugins up to date.
- ✓ Monitor and log integrations.

Remember
- Jenkins is the glue that connects all DevOps tools and automates **Build → Test → Deploy → Monitor** continuously.

Manjunatha HG

CI/CD Pipeline Workflow (Real Production Flow)

End-to-end CI/CD flow from code commit to production deployment.

1. Developer
2. Git Push
3. Webhook
4. Jenkins
5. Build
6. Test
7. Static Analysis
8. Package
9. Docker Image
10. Push Registry
11. Deploy Kubernetes
12. Production

1. Developer
- Developer writes code and commits to version control system.

2. Git Push
- Code is pushed to remote repository (GitHub/GitLab/Bitbucket).

3. Webhook
- Repository sends a webhook event to Jenkins with commit details (triggers automatically).

4. Jenkins
- Jenkins receives the webhook and triggers the pipeline job.

5. Build
- Jenkins pulls the code and compiles it using build tool (Maven/Gradle). Artifacts/compiled outputs are generated.

6. Test
- Automated tests (Unit/Integration) are executed. If any test fails, pipeline stops here.

7. Static Analysis
- Code quality is checked using tools like SonarQube. Quality Gate must pass to proceed further.

8. Package
- Application is packaged into an artifact (JAR/WAR/ZIP). This artifact is used for creating Docker image.

9. Docker Image
- Docker image is built from Dockerfile including the packaged artifact.

10. Push Registry
- Docker image is pushed to container registry (Docker Hub/ECR/GCR). Image version/tag is stored for deployments.

11. Deploy Kubernetes
- Jenkins deploys the application to Kubernetes cluster using kubectl/Helm. Updates Deployment, Service, ConfigMaps, Secrets, etc.

12. Production
- Application is now live and accessible to users in production environment. Monitoring, logging and alerting ensure stable operations.

Key Benefits
- ✓ Automated end-to-end delivery.
- ✓ Faster releases with high quality.
- ✓ Consistent & repeatable process.
- ✓ Early feedback & issue detection.
- ✓ Scalable and reliable deployments.

Best Practices
- ✓ Keep pipelines small & modular.
- ✓ Use versioned Docker images.
- ✓ Enforce quality gates.
- ✓ Store secrets in Jenkins Credentials.
- ✓ Monitor pipeline and application.

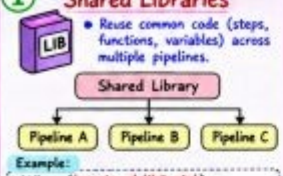
Common Mistakes
- ✗ Skipping code quality checks.
- ✗ Hardcoding credentials/secrets.
- ✗ Not handling failures properly.
- ✗ Deploying without automated tests.
- ✗ Not using versioned images.

Manjunatha HG

Advanced Jenkins Concepts

1 Shared Libraries

- Reuse common code (steps, functions, variables) across multiple pipelines.



Example:

```
@Library('my-shared-lib@main') _
pipeline {
  agent any
  stages {
    stage('Build') { steps { sayHello() } }
  }
}
```

2 Parallel Builds

- Run multiple stages/tasks simultaneously to reduce overall build time.

Example:

```
stage('Parallel Tests') {
  parallel {
    stage('Unit Test') {
      steps { sh 'mvn test' }
    }
    stage('Integration Test') {
      steps { sh './gradlew test' }
    }
    stage('UI Test') {
      steps { sh 'npm test' }
    }
  }
}
```

3 Matrix Builds

- Run the same job with multiple combinations (OS, JDK, Browser etc.).

Example:

```
matrix {
  axes {
    axis { name 'JDK'; values '17', '21' }
    axis { name 'OS'; values 'Linux', 'Windows' }
  }
  stages {
    stage('Build') {
      steps { sh 'mvn clean install' }
    }
  }
}
```

4 Credentials Management

- Store secrets (username/password, SSH key, tokens) securely in Jenkins Credentials Store.

Used for:

- ✓ Git / GitHub access
- ✓ Docker Registry login
- ✓ SSH to servers
- ✓ API tokens

Example:

```
withCredentials([
    usernamePassword(
      credentialsId: "git-creds",
      usernameVariable: "USER",
      passwordVariable: "PASS"
)]) {
  sh 'git push https://
  ${USER}:${PASS}@github.com/
  org/repo.git'
}
```

5 Environment Variables

- Store and use dynamic values across pipeline or stages.

Ways to use:

- ✓ environment block
- ✓ withEnv step
- ✓ Global / Folder / Job level

Example:

```
pipeline {
  agent any
  environment {
    APP_ENV = 'dev'
    DOCKER_REG = 'docker.io/rahash123'
  }
  stages {
    stage('Show Env') { steps { sh 'echo $APP_ENV' } }
  }
}
```

6 Parameters

- Pass custom inputs at runtime to make pipeline flexible.

Example:

```
parameters {
  string(name: 'APP_VERSION',
         defaultValue: '1.0.0')
  choice(name: 'ENV', choices: ['dev', 'qa', 'prod'])
  booleanParam(name: 'RUN_TESTS', defaultValue: true)
}
```

Use in pipeline:

```
sh "mvn deploy -Dversion=${params.APP_VERSION}"
sh "echo Environment: ${params.ENV}"
```

7 Input Approval

- Pause pipeline and wait for manual approval before proceeding.

Example:

```
stage('Approval') {
  steps {
    input message: 'Deploy to Production?',
          ok: 'Deploy',
          submitter: 'admin,devops-team'
  }
}
```

Useful for promoting builds to higher environments (QA → UAT → Prod).

8 Multibranch Pipeline

- Automatically create pipelines for each branch in SCM repository.

Benefits:

- ✓ Auto-detect branches
- ✓ Auto-create & manage jobs
- ✓ PR & branch build support
- ✓ Branch specific Jenkinsfile

Flow:



9 Pipeline Triggers

- Define when and how the pipeline should start automatically.

Common Triggers:

- ✓ SCM Push (Webhook)
- ✓ Poll SCM
- ✓ Timer / Cron
- ✓ Upstream Job Trigger
- ✓ Manual Trigger

Example (Declarative):

```
triggers {
  githubPush()
  pollSCM('H/5 * * * *')
  cron('0 2 * * *')
}
```

Best Practices

- ✓ Use Shared Libraries for reusability.
- ✓ Keep credentials secure.
- ✓ Use parameters to make pipeline dynamic.
- ✓ Enable multibranch for active repos.
- ✓ Add input step for production deployments.
- ✓ Use triggers to automate builds.

Remember

Advanced concepts make your Jenkins pipelines powerful, secure, scalable and production-ready.

Common Mistakes

- ✗ Hardcoding credentials in pipeline.
- ✗ Too many parallel stages → resource issues.
- ✗ Not using parameters → inflexible pipeline.
- ✗ Missing input approval for production.
- ✗ Overusing polling instead of webhooks.

Manjunatha HG

Jenkins Best Practices

Following best practices makes your Jenkins environment secure, maintainable, scalable and production-ready.

1 Folder Structure

- Organize jobs in folders by team or project.
- Keeps Jenkins clean and easy to manage.
- ✓ Example: /Company/Project/Service
- ✓ Use View and Blue Ocean for clarity.

2 Naming Convention

JOB-NAME

- Use clear, consistent and meaningful names for jobs, pipelines and folders.
- ✓ Format: <Project>-<Env>-<Job>
- ✓ Example: payment-dev-build

3 Least Privilege

- Give users only the permissions they need. Follow RBAC.
- ✓ Create roles with minimal access.
- ✓ Avoid using admin for daily tasks.

4 Secrets Management

- Store secrets securely using Jenkins Credentials Store.
- Use Username/Password, SSH Key, Secret Text, Secret File.
- Never hardcode credentials in Jenkinsfile.

5 Plugin Updates

- Keep Jenkins core and plugins updated for security and stability.
- Check updates regularly.
- Test in non prod before upgrading.

6 Backup Jenkins

- Backup Jenkins home directory regularly.
- Backup: $JENKINS_HOME
- Includes jobs, configs, credentials.
- Store backups in remote location.

7 Agent Separation

JENKINS — AGENT 1 — AGENT 2

- Use dedicated agents for different workloads (build, test, deploy).
- ✓ Improves performance & security.
- ✓ Use labels to route jobs to agents.

8 Pipeline Reusability

- Reuse common logic using Shared Libraries, templates and functions.
- ✓ Reduces duplication.
- ✓ Improves maintainability.

9 Pipeline as Code

JENKINSFILE { }

- Store Jenkinsfile in SCM (Git).
- Version, review and track changes.
- ✓ Enables collaboration.
- ✓ Supports rollback and audit.

10 Build Cleanup

- Clean workspace and old builds regularly to save disk space.
- ✓ Use: cleanWs(), logRotator.
- ✓ Delete old artifacts and workspaces.

11 Logs Monitoring

- Monitor Jenkins logs, build logs and system health.
- ✓ Use log aggregators (ELK, Loki).
- ✓ Setup alerts for failures.

12 General Best Practices

- Use Multibranch Pipeline for SCM projects.
- Use parameters for flexibility.
- Fail fast and notify on failures.
- Not updating plugins and processes.

Key Benefits

- ✓ Secure Jenkins environment.
- ✓ Easy to manage and scale.
- ✓ Reliable CI/CD pipelines.
- ✓ Faster builds and feedback.
- ✓ Better code quality and stability.

Pro Tip

Automate everything: Infrastructure, Builds, Tests, Deployments and Monitoring. That's the power of DevOps!

Common Mistakes

- ✗ Running builds on master node.
- ✗ Hardcoding credentials in pipelines.
- ✗ Not cleaning workspaces/builds.
- ✗ Not updating plugins regularly.
- ✗ Skipping logs and monitoring.

Manjunatha HG



Interview Questions & Common Errors

Top Interview Questions

1. **Difference between CI and CD?**
   - CI = Code integration, build & test automatically.
   - CD = Code is automatically delivered/deployed to environments.
2. **Freestyle vs Pipeline?**
   - Freestyle = UI based, hard to maintain.
   - Pipeline = Code based (Jenkinsfile), version controlled & reusable.
3. **Controller vs Agent?**
   - Controller (Master) = Schedules jobs, manages & distributes work.
   - Agent (Node) = Executes the build jobs.
4. **What is Jenkinsfile?**
   - A text file that contains the definition of a Jenkins Pipeline.
   - Stored in SCM (Git).
5. **Why Pipeline as Code?**
   - Version control, reusability, collaboration, review, auditability and easier rollback.
6. **What is Workspace?**
   - Directory on agent node where the code is checked out and build is executed.
7. **How Webhooks work?**
   - On code push, SCM sends an HTTP request to Jenkins, which starts the build automatically.
8. **What is Credentials?**
   - Jenkins secure store for usernames, passwords, tokens, SSH keys, etc.
   - Used in pipelines and jobs.
9. **What is Parallel Pipeline?**
   - Multiple stages or tasks run simultaneously to reduce total build time.
10. **What is Blue Ocean?**
    - Modern UI for Jenkins Pipelines.
    - Visualize, monitor and debug pipelines easily.

Common Errors & Fixes

1. **Build Failed**
   - Reason: Compilation or test failures.
   - Fix: Check console logs, fix code/test issues.
2. **Agent Offline**
   - Reason: Agent not connected / offline.
   - Fix: Check agent service, network & Jenkins node configuration.
3. **Permission Denied**
   - Reason: Insufficient permissions on files, folders or commands.
   - Fix: Update file permissions or use correct user.
4. **Git Authentication Failed**
   - Reason: Invalid credentials / token expired.
   - Fix: Update credentials in Jenkins Credentials Store.
5. **Maven Build Failure**
   - Reason: Dependency not found / plugin issues.
   - Fix: Check settings.xml, repository, dependencies & Maven version.
6. **Docker Permission Denied**
   - Reason: User not in docker group / socket permission issue.
   - Fix: Add user to docker group or use proper docker socket permissions.
7. **Plugin Conflict / Update Failed**
   - Reason: Incompatible plugins or old version.
   - Fix: Update plugins, restart Jenkins and check compatibility.

Best Practices

- ✓ Always use Pipeline as Code.
- ✓ Store Jenkinsfile in SCM (Git).
- ✓ Use shared libraries for reusability.
- ✓ Secure credentials and rotate regularly.
- ✓ Keep plugins updated.
- ✓ Monitor and clean old builds.

Interview Tip

- Always explain with real-world examples.
- Keep answers concise, practical and focused on Jenkins best practices.

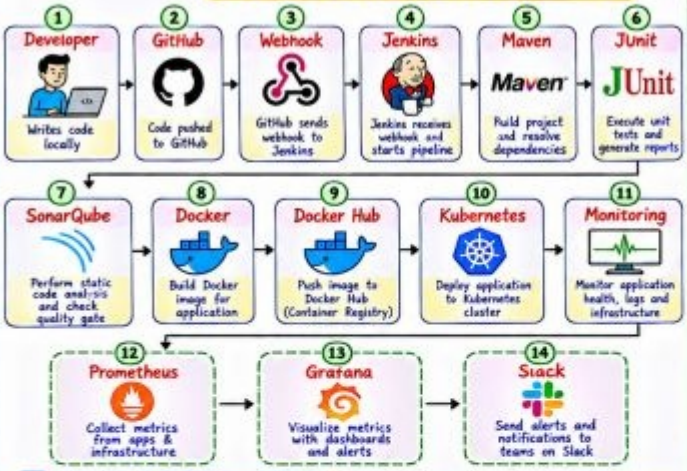
Remember

- Jenkins = Automation + Orchestration.
- Use the right tool, secure it well and keep it maintainable!

Manjunatha HG

Real DevOps Project Workflow

End-to-End Production Architecture



Tech Stack

- **Git** — Version control system
- **GitHub** — Code hosting & collaboration
- **Jenkins** — Automation server & CI/CD orchestration
- **Maven** — Build automation tool (Java)
- **Java** — Programming language
- **JUnit** — Unit testing framework
- **SonarQube** — Code quality & static analysis
- **Docker** — Containerization platform
- **Kubernetes** — Container orchestration platform
- **Prometheus** — Monitoring & metrics collection
- **Grafana** — Metrics visualization & dashboards
- **Slack** — Team communication & notifications

End-to-End Flow Explanation

1	Developer	Writes code and commits to local repository.
2	GitHub	Code is pushed to remote repository.
3	Webhook	GitHub triggers webhook on every push.
4	Jenkins	Jenkins receives the webhook and triggers the pipeline.
5	Maven	Compiles the code, runs build and manages dependencies.
6	JUnit	Executes unit tests and generates test reports.
7	SonarQube	Scans code for bugs, vulnerabilities and code smells.
8	Docker	Builds Docker image of the application.
9	Docker Hub	Stores Docker image in registry for deployments.
10	Kubernetes	Pulls image and deploys app to Kubernetes cluster.
11	Monitoring	Application health, logs and metrics are monitored.
12	Prometheus	Scrapes metrics from apps, nodes and services.
13	Grafana	Visualizes metrics with dashboards and alerts.
14	Slack	Sends build status, alerts and notifications to the team.

Key Benefits

- ✓ Fully automated CI/CD pipeline.
- ✓ Faster and reliable software delivery.
- ✓ Early feedback on code quality.
- ✓ Better collaboration and visibility.
- ✓ Scalable and production-ready architecture.

Best Practices

- ✓ Keep Jenkins, plugins and agents up to date.
- ✓ Use Pipeline as Code and store Jenkinsfile in SCM.
- ✓ Use credentials store and avoid hardcoding secrets.
- ✓ Implement quality gates using SonarQube.

Pro Tip

- ✓ Use Docker agents for consistency and isolation.
- ✓ Monitor pipelines and set up alerts (Prometheus + Slack).
- ✓ Keep build logs, artifacts and workspaces clean.
- ✓ Use Multibranch Pipeline for better development.

Common Mistakes

- ✗ Running builds on master node.
- ✗ Hardcoding credentials in pipelines.
- ✗ Not cleaning workspaces / builds.
- ✗ Not updating plugins regularly.
- ✗ Skipping logs and monitoring.

Manjunatha HG